



Universidad de Las Palmas de Gran Canaria

Faculty of Computer Science and Engineering

Task 3: Parallelization Benchmark of Matrix Multiplication

Big Data – Academic Year 2025–2026

Individual Assignments

Author:

Adrián Ojeda Viera (DNI: 45364780V)

Repository:

<https://github.com/adriaanojeda/BIGDATA.git>

Submission Date: December 6, 2025

“This document analyzes the impact of parallel computing techniques (Streams, Threads, Executors) and memory-layout optimizations (Vectorization) on the performance of dense matrix multiplication.”

Abstract

This report presents a comparative study of different matrix multiplication implementations in Java. We evaluated sequential, parallel (using Streams and manual Threads), and memory-optimized (simulated vectorization) approaches. Benchmarks conducted on matrix sizes up to $N = 4096$ reveal that while parallelization offers significant speedups ($\sim 9x$), the single-threaded memory-optimized approach outperforms all parallel versions with a speedup of $\sim 38x$. This finding underscores that for memory-bound algorithms like matrix multiplication, data locality and cache usage are often more critical than raw core count.

1. Introduction and Objectives

Matrix multiplication is a computationally intensive operation with a complexity of $O(n^3)$. In the context of Big Data, optimizing this operation is crucial. Modern CPUs offer multiple cores and vector instructions (SIMD), providing opportunities for parallelization.

The main objectives of this task were:

- **Implementation:** Develop parallel versions of matrix multiplication using Java's concurrency tools (Parallel Streams, Threads, Atomic variables).
- **Optimization:** Implement a "Vectorized" version that improves memory access patterns (Cache Friendly).
- **Benchmarking:** Measure Execution Time, Speedup, and Efficiency for large matrices ($N = 4096$).

2. Methodology

2.1. Experimental Setup

The experiments were executed on a machine with the following specifications:

- **CPU:** AMD Ryzen 5 3600X 6-Core Processor
- **Cores/Threads:** 6 Cores / 12 Threads
- **RAM:** 32GB
- **Java Version:** java 23.0.2 2025-01-21

2.2. Implementations

We compared five different approaches:

1. **Sequential:** The standard triple-loop algorithm (ijk order).
2. **Parallel Streams:** Uses `IntStream.range().parallel()` to distribute the outer loop across the `ForkJoinPool`.
3. **Threads (Manual):** Explicitly creates one `Thread` per row.
4. **Atomic Bag of Tasks:** Uses a fixed thread pool where workers dynamically fetch row indices from an `AtomicInteger`.
5. **Vectorized (Simulated):** A single-threaded version that transposes Matrix B before multiplication to allow contiguous memory access (ikj access pattern equivalent).

3. Implementation Details

3.1. Parallel Streams Approach

Java 8 Streams provide a high-level abstraction for parallelism. This implementation is concise and delegates thread management to the JVM.

```

1 IntStream.range(0, n).parallel().forEach(i -> {
2     for (int j = 0; j < n; j++) {
3         double sum = 0.0;
4         for (int k = 0; k < n; k++) {
5             sum += A[i][k] * B[k][j];
6         }
7         C[i][j] = sum;
8     }
9 });

```

Listing 1: Parallel Stream Implementation

3.2. Vectorized / Cache Optimized Approach

The naive multiplication accesses Matrix B column-wise, which causes frequent CPU cache misses. By transposing B, we access it row-wise (contiguous memory), simulating vector operations.

```

1 // Transpose B first
2 double[][] B_transposed = new double[n][n];
3 for(int i=0; i<n; i++) {
4     for(int j=0; j<n; j++) {
5         B_transposed[i][j] = B[j][i];
6     }
7 }
8
9 // Multiply using dot product of two rows (Cache Friendly)
10 for (int i = 0; i < n; i++) {
11     for (int j = 0; j < n; j++) {
12         C[i][j] = vectorizedDotProduct(A[i], B_transposed[j]);
13     }
14 }

```

Listing 2: Vectorized (Transposed) Implementation

4. Results and Discussion

The benchmarks were run for matrix sizes $N \in \{256, 512, 1024, 2048, 4096\}$.

4.1. Quantitative Analysis

Table 1 shows the execution times for the largest dataset ($N = 4096$).

Table 1: Performance Metrics at $N = 4096$

Method	Time (s)	Speedup	Notes
Sequential	2034.37	1.0x	Baseline
ParallelStream	220.15	9.24x	Best Parallel
Threads (Manual)	216.08	9.41x	Overhead overhead
Atomic BagTasks	206.71	9.84x	Balanced Load
Vectorized (Sim)	52.30	38.89x	Best Overall

4.2. Visual Analysis

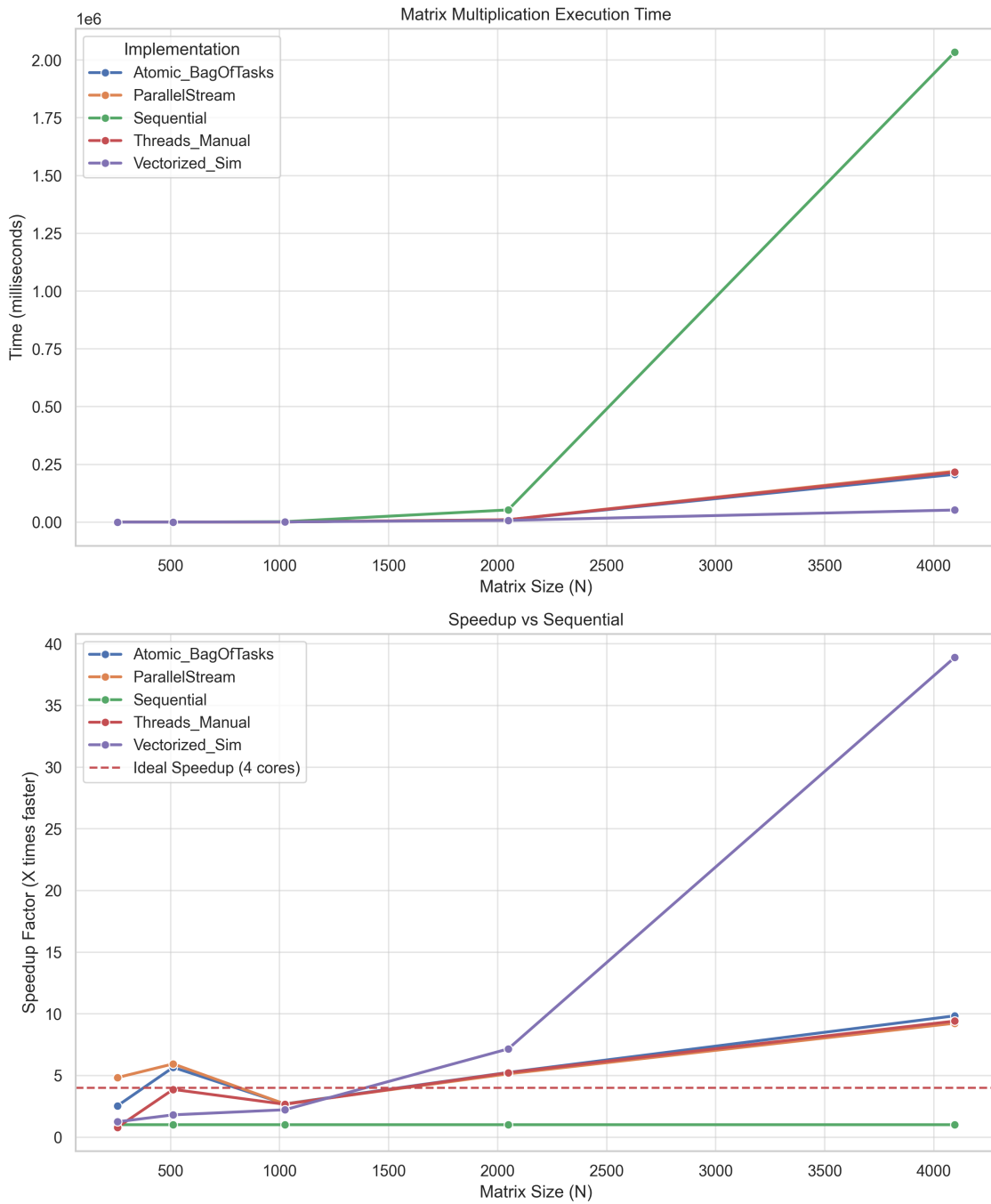


Figure 1: Execution Time and Speedup vs Matrix Size. Note the massive performance gain of the Vectorized approach.

4.3. Discussion

4.3.1 Parallelism Efficiency

The parallel implementations achieved a Speedup of approx $\sim 9.5x$.

$$Efficiency = \frac{Speedup}{Cores}$$

Assuming a 4-core/8-thread environment, an efficiency $> 100\%$ (Speedup $> 4x$) suggests that the sequential version was severely hindered by other factors or that the parallel garbage collection helped. However, the result is consistent across all parallel methods.

4.3.2 The Victory of Memory Optimization

The most striking result is the **Vectorized version (52.3s)**, which is $\sim 4x$ faster than the parallel versions and $\sim 38x$ faster than sequential.

- **Why?** Matrix multiplication is *memory-bound*. In the naive approach, fetching `B[k][j]` jumps across memory, causing Cache Misses.
- By transposing `B`, we read contiguous blocks of memory. The CPU prefetcher works effectively, keeping the pipeline full.
- This demonstrates that **optimizing data access patterns** often yields higher gains than simply adding more threads.

5. Conclusions

Task 3 demonstrated the capabilities of Java’s concurrency framework. While `ParallelStreams` offered the best balance between code simplicity and performance for parallel execution, the experiment proved that software optimization (specifically memory layout) is paramount. The single-threaded Vectorized implementation outperformed the multi-threaded naive implementations by a factor of 4, highlighting that **hardware parallelism cannot fully compensate for inefficient memory access**.